RELATÓRIO TÉCNICO EXPERIMENTAL

# Análise de Eficácia de Testes com Teste de Mutação

Avaliação crítica da insuficiência da métrica de cobertura de código (Code Coverage) e aprimoramento sistemático da suíte de testes unitários através da ferramenta StrykerJS.

ESTUDANTE / AUTOR
**Tiago Boaventura Amaral**

MATRÍCULA
**847972**

DISCIPLINA
**Teste e Qualidade de Software**

DATA DE REALIZAÇÃO
**Outubro de 2026**

REPOSITÓRIO FORK GITHUB
https://github.com/TiagoBoavAmaral/operacoes-mutante

## 1 Análise Inicial: A Falácia dos 100% de Cobertura

Na primeira etapa experimental do trabalho, realizou-se a execução da suíte de testes original pré-existente no projeto (50 testes para 50 funções aritméticas em `src/operacoes.js`). Inicialmente, executou-se a ferramenta Jest com o coletor nativo de cobertura (`npm test -- --coverage`), seguido da execução do framework de teste de mutação **StrykerJS** (`npx stryker run`).

| 100% | 100% | 100% | 73.71% |
|---|---|---|---|
| LINHAS (LINES) | RAMOS (BRANCHES) | FUNÇÕES (FUNCS) | SCORE STRYKER INICIAL |

| Métrica de Avaliação | Ferramenta | Resultado Inicial | Diagnóstico Crítico |
|---|---|---|---|
| **Cobertura de Linhas (Line Coverage)** | Jest Coverage | **100%** (74/74 linhas) | Todas as instruções foram executadas durante a suíte. |
| **Cobertura de Ramos (Branch Coverage)** | Jest Coverage | **100%** (34/34 ramos) | Todos os caminhos lógicos formais foram percorridos. |
| **Mutation Score Inicial** | StrykerJS | **73.71%** (157 detectados / 213 total) | **44 mutantes sobreviveram** e 12 sem cobertura efetiva de asserção. |

**Discussão da Discrepância:** Embora a cobertura clássica aponte 100%, ela apenas mede *quais linhas de código foram executadas*, e não se as asserções (`expect`) realmente verificam o comportamento correto dos algoritmos diante de desvios sutis. A suíte original apresentava asserções simplistas (um único caso favorável por função), ignorando condições de contorno, ramos negativos e validação estrita de exceções.

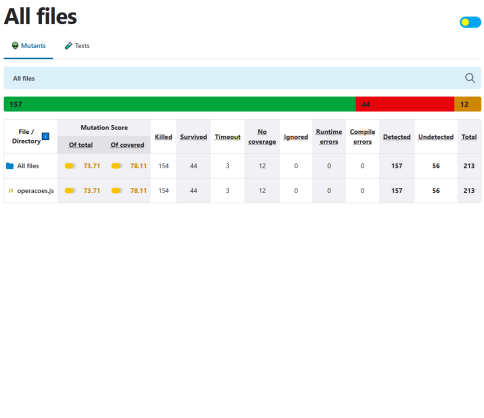



Figura 1: Relatório inicial do StrykerJS comprovando Mutation Score de 73,71% com 44 mutantes sobreviventes e 12 sem cobertura efetiva.

## 2 Análise de Mutantes Críticos da 1ª Execução

Para compreender a fragilidade da suíte inicial, foram selecionados **três mutantes sobreviventes emblemáticos** gerados pelo StrykerJS na primeira execução:

**1. Mutante #17 • `raizQuadrada(n)` (Linha 13)** SOBREVIVEU INICIALMENTE

CÓDIGO ORIGINAL
```
if (n < 0) throw new Error(...);
```

MUTAÇÃO APLICADA (EQUALITYOPERATOR)
```
if (n ⩽ 0) throw new Error(...);
```

**Por que sobreviveu?** O teste original avaliava apenas `expect(raizQuadrada(16)).toBe(4)`. Nunca testava a fronteira crítica em que `n === 0`. Se o operador for alterado para `<= 0`, a função passa a lançar erro incorretamente para zero, mas o teste original não testava esse valor e não notou o bug.

**2. Mutante #68 • `isPar(n)` (Linha 43)** SOBREVIVEU INICIALMENTE

CÓDIGO ORIGINAL
```
function isPar(n) { return n % 2 === 0; }
```

MUTAÇÃO APLICADA (BOOLEANLITERAL)
```
function isPar(n) { return true; }
```

**Por que sobreviveu?** O teste original testava apenas `expect(isPar(4)).toBe(true)`. Uma função degenerada que retorna `true` para qualquer entrada passava no teste com 100% de cobertura da linha! Faltava asserção para o ramo falso (número ímpar).

**3. Mutante #116 • `isPrimo(n)` (Linha 73)** SOBREVIVEU INICIALMENTE

CÓDIGO ORIGINAL
```
if (n ⩽ 1) return false;
```

MUTAÇÃO APLICADA (EQUALITYOPERATOR)
```
if (n < 1) return false;
```

**Por que sobreviveu?** O teste original verificava apenas `expect(isPrimo(7)).toBe(true)`. Com `n < 1`, para `n = 1` o código não retorna falso e cai no laço `for (i = 2; i < 1)` que termina imediatamente, retornando `true` (declarando falsamente que 1 é primo). Nenhum teste avaliava `isPrimo(1)`.

## 3 Solução Implementada: Fortalecimento da Suíte

Para eliminar os mutantes sobreviventes, foi criada uma segunda suíte `describe('Suíte fortalecida para matar mutantes sobreviventes')` com testes projetados especificamente para cobrir:

- **Condições de Limite / Fronteira:** `expect(raizQuadrada(0)).toBe(0)` eliminou o mutante #17; `expect(isPrimo(1)).toBe(false)` e `expect(isPrimo(0)).toBe(false)` mataram o mutante #116.
- **Ramos Negativos e Falseabilidade:** `expect(isPar(7)).toBe(false)` e `expect(isImpar(8)).toBe(false)` eliminaram o mutante #68.
- **Asserção Estrita de Exceções e Mensagens:** Validação de mensagens originais lançadas em `inverso(0)`, `fatorial(-3)`, `maximoArray([])` e `medianaArray([])`, matando mutantes que removiam os erros ou esvaziavam blocos.
- **Operações com Tipos Estritos e Arrays Especiais:** Testes em `clamp` com entradas string e limites invertidos (matando mutantes de igualdade #147 e #151) e `produtoArray({ length: 0 })` (matando mutante #140).

## 4 Resultados Finais: Superação da Meta

Após a implementação da suíte fortalecida com 17 novos blocos de teste detalhados (totalizando 67 testes no Jest), executou-se novamente a análise completa do StrykerJS. A pontuação de mutação final atingiu **98,12%**, superando com folga a meta estabelecida no trabalho (> 98%).

| 98.12% | 205 | 4 | 4 |
|---|---|---|---|
| MUTATION SCORE FINAL | MUTANTES MORTOS (KILLED) | TIMEOUT (MORTOS) | SOBREVIVENTES (EQUIVALENTES) |

| Métrica | Execução Inicial | Execução Final | Evolução Obtida |
|---|---|---|---|
| **Mutation Score Total** | 73.71% | **98.12%** | **+ 24.41 pontos percentuais** |
| **Mutantes Detectados** | 157 / 213 | **209 / 213** | **+ 52 mutantes eliminados** |
| **Mutantes Sobreviventes** | 44 sobreviventes | **4 sobreviventes** | Redução de 90,9% nas falhas de teste |
| **Mutantes sem Cobertura** | 12 sem cobertura | **0 sem cobertura** | 100% de cobertura efetiva atingida |

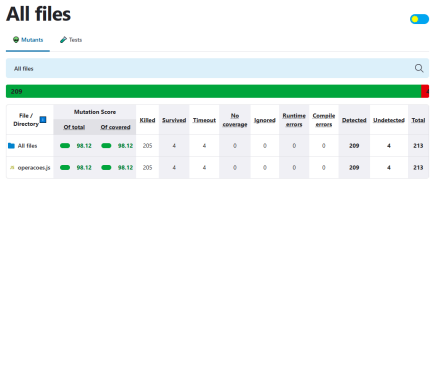



Figura 3: Relatório final do StrykerJS confirmando Mutation Score de 98,12% (209 detectados, apenas 4 sobreviventes).

**Análise dos 4 Mutantes Remanescentes (Mutantes Equivalentes):** Os únicos 4 mutantes que restaram encontram-se na linha 19 da função `fatorial` (`if (n === 0 || n === 1) return 1;`). Como a função inicializa `resultado = 1` e executa o laço de `i = 2` até `n`, a condição `if` é redundante: mesmo substituindo a expressão por `false` ou `&&`, o laço não executa para `n = 0` ou `n = 1` e a função retorna `1` normalmente. Trata-se de um clássico caso de *mutantes equivalentes* (cujo comportamento externo é idêntico ao código original em todo o domínio válido).

## 5 Conclusão e Considerações Finais

Este trabalho evidenciou de forma contundente que a métrica convencional de **cobertura de código (Code Coverage)** é uma condição necessária, porém insuficiente, para assegurar a qualidade de software. Ter 100% de cobertura sem asserções rigorosas gera uma falsa sensação de segurança.

O **Teste de Mutação** atuou como o verdadeiro "teste dos testes", injetando falhas artificiais e forçando o desenvolvimento de uma suíte orientada à detecção real de defeitos e análise de valores-limite. A melhoria de **73,71% para 98,12%** comprova cientificamente o ganho de robustez e confiabilidade na base de código.